CNN-Based Violence Detection: Evaluation and Experiments

Iulian Vlascu
Faculty of Mathematics and Informatics
Transilvania University of Braşov
Braşov, Romania
constantin.vlascu@student.unitbv.ro

Alexandra Băicoianu
Faculty of Mathematics and Informatics
Transilvania University of Braşov
Braşov, Romania
a.baicoianu@unitbv.ro

2025 18th International Conference on Engineering of Modern Electric Systems (EMES) | 979-8-3315-2577-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/EMES65692.2025.11045585

***Abstract*—Given the growing reliance on surveillance cameras and the increasing need for automatic monitoring of violent activities, various deep learning-based approaches have been developed to address these challenges. In this study, we started from two existing methods for violence detection in video streams based on convolutional neural networks, aiming to evaluate their performance and to investigate potential improvements. Reproducing these methods in our experimental setting allowed us to gain deeper insights into their implementation challenges, computational efficiency, and adaptability to various scenarios. The first approach employs an enhanced 3D CNN with a compact architecture inspired by DenseNet elements for spatiotemporal feature extraction, while the second method explores a more complex and computationally intensive architecture. We assessed the strengths and limitations of these models and to identified directions for optimizing the detection pipeline for real-world applications.**

***Keywords*—*violence detection*, *CNN*, *DenseNet*, *Keras*.**

I. Introduction

The growing prevalence of surveillance systems in both public and private spaces has led to the generation of vast amounts of video data, with the number of CCTV cameras estimated at approximately 1 billion in 2020 [1]. As urban safety becomes a critical concern, monitoring systems for aggressive behavior have become essential for ensuring security across cities. However, one of the primary challenges in developing such systems is the processing of this massive volume of data. Manual review is not only impractical due to the sheer amount of footage, but it is also prone to human error, making it inadequate for real-time analysis. Consequently, researchers have explored various approaches to develop reliable, accurate, and cost-effective violence detection systems. Early studies in this field focused on handcrafted, feature-based methods, which typically combined feature extraction with classifiers such as Support Vector Machines (SVMs) and others [2], [3]. These classifiers used the extracted features to predict whether the input video depicted violent or non-violent behavior. To capture aggressive actions and distinguish between violent and non-violent events both spatially and temporally, the feature extraction process relied on visual cues, including motion trajectories, optical flow, and texture analysis. For example [4], methods based on hierarchical hidden Markov models (HHMM) and specialized descriptors like violent flow features provided initial frameworks for violence detection. Such approaches present some limitations due to the complexities of real-world surveillance scenarios, such as the variation of view angles and the uncontrolled and diverse crowd dynamics. Such complexities moved the researchers attention to developing more adaptive techniques that could learn more robust patterns from data. In the recent studies, convolutional neural networks (CNNs), and in particular 3D CNN architectures, have started to be the go-to choice for the violence detection task because they have shown remarkable ability in capturing spatiotemporal patterns directly from video sequences due to their deep connecting layers that manage to extract small details from every corner of a video frame. These models outperform in every metric the earlier handcrafted methods due to their deep layers that can be trained to learn to detect on their own the challenging temporal and spatial patterns of violence, achieving superior accuracy and robustness in complex scenarios.

Among the existing literature, references [5] and [6] were identified as especially relevant to the scope of this research, for several reasons discussed below. First of all, the results of these two studies appeared highly promising. The authors conducted comparisons not only with classical approaches involving feature extraction followed by SVMs, but also with neural network-based methods, particularly recurrent neural networks such as LSTM. These comparisons demonstrated that the two proposed methods outperformed the others, achieving significantly better accuracy and loss metrics. Furthermore, the technologies presented in the studies are more modern, relying on CNN architectures. These architectures have gained considerable traction in recent years due to their superior efficiency. Unlike earlier approaches, which required manual feature extraction and complex computer vision algorithms, CNN-based methods automate much of this process, offering improved performance and scalability. Second, we identified potential for improvement through the inclusion of more detailed results. As will be discussed in the following sections, one of the main contributions of this study lies in the introduction of additional evaluation metrics. These metrics aim to provide a deeper understanding of the behavior of the two architectures and to support a more thorough comparison of their performance, particularly in terms of their suitability for integration into real-world systems. Moreover, the two

979-8-3315-2577-4/25/$31.00 ©2025 European Union

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 27,2026 at 02:28:47 UTC from IEEE Xplore. Restrictions apply.

referenced studies focus exclusively on the same three datasets. To expand on their work, we extended the evaluation by testing one of the models on a different yet similarly complex dataset, as detailed in the current study. Lastly, another reason for selecting these two studies is the complexity of the proposed architectures. Their high computational requirements offer an opportunity to explore the potential limitations related to training and testing such models on less powerful systems. This aspect also contributes to assessing their practicality in various deployment scenarios.

II. Datasets For Violence Detection

The intricate nature of violence detection has driven the creation of a variety of datasets which are widely utilized in the literature for a range of deep learning tasks. In this section we provide a brief overview of the most commonly used datasets for violence detection, discussing their respective advantages and limitations.

A. Violent Crowd Dataset (VSD)

Violent Crowd Dataset consists of 246 video clips (320×240) pixel resolution sourced from YouTube, specifically designed to capture dynamic crowd behaviors that include both violent and non-violent interactions. The videos are categorized into two balanced classes: 123 clips correspond to violent events, and 123 to non-violent events [2].

One of the key advantages of this dataset is its ability to represent complex scenarios, with diverse scenes that reflect real-life situations where crowd dynamics are inconsistent. This variability allows models trained on this dataset to potentially generalize better in practical applications. Additionally, the dataset's balanced class distribution of violent and non-violent clips mitigates the issue of class imbalance, which is often encountered during training, ensuring a more reliable model performance. However, the dataset does have some limitations. The relatively low video quality may pose challenges in extracting accurate temporal and spatial features, which can affect the model's ability to capture fine-grained patterns of violent behavior. Furthermore, some clips are brief in duration, potentially failing to capture the full temporal progression of violent events, such as extended altercations that might be fragmented into multiple clips.

In Fig. 1 we can see two examples from the Violent Crowd Dataset II-A, one with a crowd fight and one with a normal interaction in a crowd.



Crowd Violence Crowd Non Violence

Fig. 1. Samples from Violent Crowd Dataset

B. Violence in Movies Dataset (VMS)

The Violence in Movies dataset comprises 200 (360×250) resolution video clips, with a maximum of 50 frames per video, sourced from action movies for fight scenes and from various public action recognition datasets for non-violent scenes. The dataset has been previously used in [7]. This dataset is designed to simplify the violence detection task by focusing on scenes with minimal or no camera motion, thereby reducing the noise typically introduced by camera movement.

An advantage of this dataset is the reduced camera motion, which is crucial for isolating the subjects in the frame. This allows deep learning models to more effectively and accurately learn the movement patterns of the subjects involved in the scenes. Furthermore, the variety of scenes sourced from different videos enhances the dataset's diversity, helping models generalize better to a broader range of fight scenarios. However, the dataset has some limitations. The 50-frame length restriction may cause the model to miss the full context of violent events, as it limits the capture of long-term temporal patterns in the behavior of the subjects. Additionally, the relatively low resolution of the videos can impact the extraction of crucial features from the spatio-temporal patterns, which are essential for accurate violence detection.

In Fig. 2 we can see two examples from the Movies Dataset II-B, one with a fight scene and one with a person doing some exercise.



Movies Violence Movies Non Violence

Fig. 2. Samples from Movies Dataset

C. Hockey Fight Dataset (HFD)

The Hockey Fight dataset consists of 1000 (360×288) resolution video clips sourced from the National Hockey League (NHL) [7], with 500 clips labeled as fights and 500 as non-fights. Each video clip contains 50 frames. One of the key strengths of this dataset lies in its consistent context, as both the fight and non-fight scenes occur within the same hockey environment, providing a unified setting for model training.

An advantage of this dataset is its consistent environment, which helps reduce potential background biases that could influence the model's learning process. By maintaining the same context across both fight and non-fight clips, it enhances the model's ability to focus on the relevant features of violence detection. Despite its advantages, the dataset has some limitations. One drawback is the limited potential of generalization. Since the dataset is specific to hockey and maintains a consistent environment, the model trained on it may fail to

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 27,2026 at 02:28:47 UTC from IEEE Xplore. Restrictions apply.

detect violence in different contexts, where the environment and interactions could differ significantly. Additionally, the 50-frame clip length may limit the model's ability to capture extended temporal patterns of violent behavior, restricting its understanding of longer sequences of actions.

In Fig. 4 are two examples from the Hockey Fights Dataset II-C, one with a fight between two players and one with a normal play.



Hockey Violence Hockey Non Violence

Fig. 3. Samples from Hockey Fights Dataset

D. Real Life Violence Situations (RLVS)

The dataset presented in [8] comprises of 2000 video clips with the width of the videos ranging between 224 and 1920, while the height of the videos are ranging between 224 and 1080 with an average video size of 397×551, thus having the resolution between 240p and 720p. The videos are evenly divided into 1000 violent and 1000 non-violent instances. The videos contain a diverse range of environments, including street fights, prison altercations, and school fights, featuring various individuals and contexts. Additionally, the dataset features variations in both video duration and resolution, making it more representative of real-world scenarios.

One of the key advantages of this dataset is its diversity. The inclusion of scenes captured in multiple locations, under different lighting conditions, and involving a variety of individuals contributes to better model generalization. Furthermore, the extended duration of the clips, ranging from 5 to 7 seconds, enables more effective extraction of temporal features and ensures that the model is not limited to analyzing only very short video segments. However, the dataset also presents certain limitations. The increased length of the video clips significantly raises computational requirements, particularly in terms of GPU memory usage. This could create challenges for training on systems with limited computational resources, potentially limiting accessibility for researchers with less powerful hardware.

In Fig. 4 are two examples from the Real Life Violence Situations Dataset II-D, one with a fight between two people and one with a arrow shooting competitor.

E. Custom Mix Dataset (Mix)

The custom Mix dataset later on in our study was created by combining three distinct datasets: VDS, VMS, and HFD, resulting in a total of 1446 videos. Each of these videos contains both positive and negative examples, randomly selected in equal proportions to ensure representativeness and diversity of the data.



RLVS Violence RLVS Non Violence

Fig. 4. Samples from Real Life Violence Situations Dataset

As the dataset is composed of three different sources, it benefits from the characteristics of each individual dataset but also presents certain challenges. This custom mixed dataset was chosen to ensure comprehensive coverage of test scenarios and to support the generalization of the models tested in our study, ensuring that the conclusions drawn are meaningful and applicable across a wide range of cases.

III. Reproducing Existing Experiments

Various studies in the literature propose deep learning-based solutions for violence detection in video streams. As a starting point for our research, we selected and reproduced the approach presented in [5], which outlines a method for detecting violent events using convolutional neural networks. The first step in our implementation involved preprocessing the input data, following the data processing and augmentation techniques detailed in [9], the foundational study for [5]. These preprocessing methods include an initial frame resizing, followed by data augmentation through random cropping and random horizontal flipping with a 50% probability.

Subsequently, we constructed the same architecture as described in the original studies. The input size remained consistent at (16, 112, 112, 3), where 16 is the number of frames in a single sequence (temporal dimension), the next two values are the heigth and the width of the input frames and 3 is the number of channels (RGB), and the implementation was carried out using the *Keras* library. The architecture is presented in Table I.

TABLE I. 3D CNN architecture

Layer Name	Filters	Kernel Size	Stride	Activation
Conv3D	64	(3, 3, 3)	(1, 1, 1)	ReLu
MaxPooling3D	-	(2, 2, 2)	(2, 2, 2)	-
Conv3D	128	(3, 3, 3)	(1, 1, 1)	ReLu
MaxPooling3D	-	(2, 2, 2)	(2, 2, 2)	-
Conv3D	256	(3, 3, 3)	(1, 1, 1)	ReLu
Conv3D	256	(3, 3, 3)	(1, 1, 1)	ReLu
MaxPooling3D	-	(2, 2, 2)	(2, 2, 2)	-
Conv3D	512	(3, 3, 3)	(1, 1, 1)	ReLu
Conv3D	512	(3, 3, 3)	(1, 1, 1)	ReLu
MaxPooling3D	-	(2, 2, 2)	(2, 2, 2)	-
Conv3D	512	(3, 3, 3)	(1, 1, 1)	ReLu
Conv3D	512	(3, 3, 3)	(1, 1, 1)	ReLu
MaxPooling3D	-	(2, 2, 2)	(2, 2, 2)	-
Flatten	-	-	-	-
Fully Connected	4096	-	-	ReLu
Fully Connected	4096	-	-	ReLu
Fully Connected	2	-	-	Softmax

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 27,2026 at 02:28:47 UTC from IEEE Xplore. Restrictions apply.

The final step involved training the model using the same parameters as those specified in the original study. Specifically, we employed the SGD optimizer with an initial learning rate of 10^{-3}, the loss function used was sparse categorical crossentropy, and the learning rate was set to halve after a predefined number of epochs.

Unfortunately, the results obtained were not entirely comparable to those reported in the original study, indicating certain discrepancies in performance. Given these less-than-optimal results, we decided to further investigate the subject by considering a more comprehensive study on violence detection. Thus, we proceeded with the reproduction of the second study [6], which follows the same methodological steps as the previous reproduction, allowing for further evaluation and comparison.

The experiments for the second study [6] began with the data augmentation process. Unlike the previous study, the frames were not resized, and instead, the augmentation strategies were applied directly to the original frames. Random cropping and random flipping are also used, but in the validation phase central crop is used instead of a random crop. In addition, a temporal random slicing is also used to extract random sequences of frames from all the frames.

We used the same architecture, being a DenseNet model it consists of more compact parts and has a lower number of trainable parameters than the previous architecture. The input size is the same (16, 112, 112, 3) and the library used is also *Keras*. The architecture employed in this study is based on a 3D DenseNet CNN, which integrates dense blocks and transition blocks to enhance the feature extraction and processing capabilities.

Further details of the DenseNet 3D CNN architecture, including specific configurations for the dense blocks and transition blocks, are provided in Tables II, III, and IV. These tables outline the parameters and structure of each component in more detail, supporting the implementation of the architecture for violence detection tasks.

TABLE II. DenseNet-like 3D CNN

Layer Name	Filters	Kernel Size	Stride	Activation
Conv3D	64	(7, 7, 7)	(1, 2, 2)	ReLu
MaxPooling3D	-	(1, 2, 2)	(1, 2, 2)	-
$DenseBlock \times 6$	-	-	-	ReLu
Transition	128	(1, 1, 1)	(2, 2, 2)	-
$DenseBlock \times 12$	-	-	-	ReLu
Transition	256	(1, 1, 1)	(2, 2, 2)	-
$DenseBlock \times 24$	-	-	-	ReLu
Global AvgPooling3D	-	-	-	-
FC	-	-	-	Softmax

Following the training procedure outlined in the study, we used the SGD optimizer with a learning rate of 10^{-3}, a momentum of 0.5, and a weight decay of 10^{-3}. In contrast to the previous training setup, we did not implement a learning rate schedule that halves over time.

Also, unfortunately, the results were not the same as in the original study, noting an observable difference in the accuracy values between the four datasets used, differences that could have occurred due to differences either in the hyperparameters or due to the system's configurations used.

TABLE III. Dense block architecture

Layer Name	Filters	Kernel Size	Stride	Activation
BatchNorm	-	-	-	-
ReLu	-	-	-	ReLu
Conv3D	$4 \times growth_rate$	(1, 1, 1)	(1, 1, 1)	-
BatchNorm	-	-	-	-
ReLu	-	-	-	ReLu
Conv3D	growth_rate	(3, 3, 3)	(1, 1, 1)	-
Concatenate	-	-	-	ReLu

TABLE IV. Transition block architecture

Layer Name	Filters	Kernel Size	Stride	Activation
BatchNorm	-	-	-	-
ReLu	-	-	-	ReLu
Conv3D	filters // 2	(1, 1, 1)	(1, 1, 1)	-
AveragePooling3D	-	(3, 3, 3)	(2, 2, 2)	-

IV. Proposed Improvements and Variations

In this section, we highlight the modifications and improvements introduced to the experiments discussed in Section III. These changes aim to address certain limitations observed in the previous experiments and enhance the overall performance and applicability of the models. By incorporating these adjustments, we sought to refine the methodology and improve the robustness of the results.

For data processing, we observed that some videos from the Hockey dataset contained black edges. To address this, we removed these black borders using *ffmpeg* processing. This ensures that when the frames are further processed and cropped, they do not contain any black regions, thus preventing potential negative impacts on the performance during training.

Fig. 5 illustrates the differences between the original, unprocessed frame and the processed frame after the removal of black bars. It is evident that the black areas occupied a significant portion of the frame, which could have seriously affected the model training process, resulting in much weaker results.



Frame with black bars Frame without black bars

Fig. 5. Example of the *ffmpeg* processing

Additionally, during the training phase, we encountered instances where the models would freeze before completing all epochs. To mitigate this issue, we introduced an early stopping

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 27,2026 at 02:28:47 UTC from IEEE Xplore. Restrictions apply.

mechanism. This mechanism halts the training process if the model's performance stagnates over a predefined number of epochs. We set a patience parameter of 20 epochs and chose validation accuracy as the monitored metric.

For the first study, we introduced additional metrics beyond those originally outlined. Specifically, we incorporated the number of FLOPs and the processing speed of a single frame across various batch sizes. These metrics enable a more comprehensive analysis of model optimizations, helping us identify the most efficient versions of the model—versions that can be effectively deployed in real-world systems. To collect this data, we utilized the profiling tools provided by TensorFlow, specifically the $tensorflow.python.profiler.model_analyzer$.

For the second study, we also introduced supplementary metrics to evaluate the quality of the model's predictions. In addition to the existing metrics, we included the loss values across the three base datasets, as well as the AUC (Area Under the Curve) value and the number of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). These additional metrics allow for a more nuanced assessment of the model's performance.

As an additional modification, we incorporated a new dataset, specifically Real Life Violence Situations Dataset II-D described in Section II, into the experiments associated with both studies mentioned. This inclusion was intended to further assess the generalizability and robustness of the models by providing a broader range of data for training and evaluation.

V. Experimental Results and Analysis

We present the experimental results obtained from our replicated experiments, compared to those reported in the original studies. Additionally, we will introduce the new metrics we have developed, which provide further insight into the model's performance and optimization. The results are displayed in the following tables, allowing for a detailed comparison and analysis of the different configurations.

In Table V we observe that the accuracy results are similar; however, the recreated model exhibits some overfitting. When comparing the AUC values, it becomes apparent that the recreated model tends to make random predictions, which contrasts sharply with the results presented in the original study. Additionally, the discrepancies in the loss values indicate that the quality of the predictions is lower than expected.

TABLE V. Results of the first study reproduction

Datasets	VCD	HFD	VMD
ACC% original	98%	99.9%	96%
AUC original	0.970	0.98	0.997
LOSS original	1.89×10^{-9}	5.77×10^{-4}	5.4×10^{-4}
ACC% our	100% train 86% val	98% train 90% val	100% train 99% val
AUC our	0.54	0.48	0.51
LOSS our	5.8×10^{-2} train 8.4×10^{-1} val	1.3×10^{-1} train 5.8×10^{-1} val	6.1×10^{-1} train 6.2×10^{-1} val

Also, the number of GFLOPs of the first model is 38.55 and the single frame processing speeds for a batch size of 1 and 8 are 4.82 ms and 2.03 ms, respectively. Comparing with the 10.4 GFLOPs of the second model and the processing speeds of 5.06 ms, 1.87 ms and 1.44 ms for a batch size of 1, 8 and 16, respectively, we can see clear performance differences, which improved the capability of the second model to be integrated into a real system. Also, these performances can be further reduced by implementing some quantization strategies.

TABLE VI. Results of the second study reproduction

Datasets	VCD	HFD	VMD	Mix	RLVS
ACC% original	97.17%	98.3%	100%	99.32%	-
ACC% our	82% train 76% val	90% train 91% val	92.9% train 85.7% val	94.1% train 73.1% val	90.5% train 79.7% val
AUC our	0.79	0.97	0.89	0.89	0.86
LOSS our	5.5×10^{-1} train 6.5×10^{-1} val	1.5×10^{-1} train 2.9×10^{-1} val	4.4×10^{-1} train 4.7×10^{-1} val	1.6×10^{-1} train 7.3×10^{-1} val	2.4×10^{-1} train 6.9×10^{-1} val

We can see in the Table VI that here the differences in accuracy are slightly greater, but we can observe from the AUC metrics that the quality of the predictions is much better because the model no longer makes random predictions. Having lower overfitting compared to the first model makes integration into a real system more plausible because this violence detection task requires a much higher quality of predictions to mitigate the risk of false predictions as much as possible.

TABLE VII. Confusion matrix values

Datasets	VCD	HFD	VMD	Mix	RLVS
TP	211	223	65	302	2053
FP	17	27	27	260	233
TN	87	235	103	481	984
FN	70	15	1	27	476

The values presented in Table VII help us see in much more detail where the model fails to provide correct predictions and where it succeeds in producing correct predictions. In order to have a more implementable model, we must pay attention to the balance between false positive and false negative predictions because these two will determine the actions that will be taken based on the predictions. A large number of false positives will overload the emergency services more, which is not really desirable, but a large number of false negatives will put people's lives in greater danger by not alerting the emergency services when it is really needed.

The results of the previously discussed experiments, along with the figures, highlight several contributions. First, in the pre-processing step presented in Fig. 5, the removal of black bars from the frames is demonstrated. This process enhances the stability of the training by eliminating noise from the input data. Second, by adding processing speed and GFLOPS metrics for the second model, we can see how the first model executes a much higher number of GFLOPs, this can also lead to a very high strain on the system on which it is trained and tested. Having the second model which has 73.01% fewer GFLOPs than the first, makes it possible to train and test on more systems. Also, this difference facilitates the local integration of the second model into a practical security system, which could result in faster response times when

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 27,2026 at 02:28:47 UTC from IEEE Xplore. Restrictions apply.

reacting to actual threats. Finally, by adding the confusion metrics: true positive, false positive, true negative and false positive to the second model as presented in Table VII, we can better appreciate the balance that we will need to have between them so that this model behaves optimally in more real-life situations. These metrics will allow us to fine tune the model so that it can respond differently based on what systems it will be integrated into and what situations it will need to respond to.

We also acknowledge the limitations present in our study, which may be attributed to system differences and variations in hyperparameters. Each model exhibits slightly different behavior depending on the system on which it is being trained. These limitations could be addressed in future studies, such as ablation studies, which would provide insights into how the components and parameters interact, and how removing certain elements might reduce system constraints and improve performance.

VI. Conclusions

Detecting violence in video streams is an area of significant interest, particularly given its potential applications in public safety and security systems. As a result, we set out in this study to explore the behavior of models proposed in [5] and [6]. By replicating the experiments and analyzing the performance of these models, we aim to understand their strengths and limitations, evaluate their performance in the context of our own experimental settings, and ultimately contribute to improving the accuracy and efficiency of such systems.

The first model, characterized by its larger and more complex architecture, demonstrated higher accuracy; however, this came at the expense of significantly increased computational demands, potentially limiting its practical deployment in resource-constrained real-world surveillance systems. In contrast, the second model, which utilized an enhanced 3D CNN architecture inspired by DenseNet elements, exhibited superior computational efficiency and faster inference times. These attributes make it more suitable for real-time applications where rapid processing is crucial.

We used new metrics to enhance the evaluation of model performance. For the first replicated experiment, we introduced metrics such as the number of FLOPs and the processing time per frame. For the second study, we expanded the analysis by incorporating AUC scores, confusion matrix classifications (including the number of true positives, false positives, true negatives, and false negatives), and loss values for each dataset individually. These additional metrics provide a more comprehensive assessment of the predictive performance of the models.

Furthermore, we made improvements to the data preprocessing pipeline by removing black edges from the videos, thereby eliminating potential biases that could affect model training. Additionally, to address the issue of training freezes, we implemented an early stopping mechanism, ensuring that the training process could be halted if model performance stagnated over a certain number of epochs. These enhancements were made to ensure a more robust and accurate evaluation of the models under consideration.

Additionally, we tested the models on a different dataset than the one used in the original studies. This alternative dataset, being more varied, allowed us to assess the models' performance in a broader range of scenarios. By using a different dataset, we aimed to better understand the models' ability to generalize across diverse contexts, providing a more comprehensive evaluation of their effectiveness beyond the original testing conditions.

Several directions for improvement can be further investigated, such as expanding datasets with new data, improving model architecture by performing ablation studies, we can also come up with improvements to the data augmentation method, we can replace the random crop that can bring imbalances to the dataset with a more intelligent crop, a crop that can use an already pre-trained model to determine the optimal crop portions of a frame.

Acknowledgement

This work was funded by the Romanian Ministry of Research, Innovation and Digitalization, SOLUTIONS project entitled "Platform for fusion and management of multi-source data collections exploitable by artificial intelligence models for predictive estimation and analysis of risk situations (AI4RISK)", contract no. 3Sol(T3)/02.09.2024.

References

[1] "Surveillance Cities: who has the most CCTV cameras in the world?" https://surfshark.com/surveillance-cities, accessed: 2025-01-10.

[2] T. Hassner, Y. Itcher, and O. Kliper-Gross, "Violent flows: Real-time detection of violent crowd behavior," in *2012 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 2012, pp. 1–6.

[3] J.-F. Huang and S.-L. Chen, "Detection of violent crowd behavior based on statistical characteristics of the optical flow," in *2014 11th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*, 2014, pp. 565–569.

[4] N. Nguyen, D. Phung, S. Venkatesh, and H. Bui, "Learning and detecting activities from movement trajectories using the hierarchical hidden markov model," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 2, 2005, pp. 955–960 vol. 2.

[5] F. U. M. Ullah, A. Ullah, K. Muhammad, I. U. Haq, and S. W. Baik, "Violence detection using spatiotemporal features with 3d convolutional neural network," *Sensors*, vol. 19, no. 11, 2019. [Online]. Available: https://www.mdpi.com/1424-8220/19/11/2472

[6] J. Li, X. Jiang, T. Sun, and K. Xu, "Efficient violence detection using 3d convolutional neural networks," in *2019 16th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 2019, pp. 1–8.

[7] E. Bermejo Nievas, O. Deniz Suarez, G. Bueno García, and R. Sukthankar, "Violence detection in video using computer vision techniques," in *Computer Analysis of Images and Patterns*, P. Real, D. Diaz-Pernil, H. Molina-Abril, A. Berciano, and W. Kropatsch, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 332–339.

[8] M. M. Soliman, M. H. Kamal, M. A. El-Massih Nashed, Y. M. Mostafa, B. S. Chawky, and D. Khattab, "Violence recognition from videos using deep learning techniques," in *2019 Ninth International Conference on Intelligent Computing and Information Systems (ICICIS)*, 2019, pp. 80–85.

[9] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3d convolutional networks," in *2015 IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 4489–4497.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 27,2026 at 02:28:47 UTC from IEEE Xplore. Restrictions apply.